

LifeFlow Blood Matching Simulation

Java Swing • OOP • File I/O • Emergency-first
matching

The problem LifeFlow addresses

A small but meaningful simulation

Blood donation scenarios involve several connected decisions: donor eligibility, unit validity, request priority, and stock availability.

Treating each part as an isolated record misses the real workflow and gives weak evidence of OOP.

LifeFlow joins the steps into one repeatable simulation that can be demonstrated live.

The intended outcome

Register donors and blood units, queue regular and emergency requests, then process the highest-priority request safely.

Persist every successful operation so the next session continues from the same state.

Four pillars define the system

01 Eligibility

Check age, weight, donation interval, and future dates before accepting a new unit.

02 Inventory

Track every unit by type, expiry, and AVAILABLE or USED status.

03 Priority

Process emergency requests first, then the oldest request at equal priority.

04 Dashboard + UX

Expose live stock, urgent demand, quick actions, and safe editing in one clear interface.

One dashboard guides the complete workflow

SEE STATUS

ACT

MATCH & SAVE

Live operational view

Review donors, available stock, pending requests, and the emergency queue at startup.

Focused quick actions

Open a clean add dialog for donors, units, or requests without leaving the dashboard.

Atomic result

Use matching units only when the full quantity is available, then persist.

The architecture stays intentionally simple

Presentation

Sidebar + CardLayout
Five focused panels
Reusable UI theme

Controller + domain

LifeFlowController
Request hierarchy
Inventory + matching

Persistence

FileManager
Three text files
Clear line-level errors

OOP concepts are visible in real behavior

The request hierarchy is the main OOP example

BloodRequest holds shared state and declares `getPriority()` as an abstract operation.

RegularRequest and EmergencyRequest override that operation with different priorities.

Abstraction + Inheritance

Private fields preserve encapsulation.

Shared request data lives once in the parent.

Subclasses supply only distinct behavior.

Runtime Polymorphism

Requests share one `ArrayList<BloodRequest>`.

Matching calls `getPriority()` through the parent.

The actual subclass decides the result at runtime.

Matching is priority-aware and atomic

Selection policy

1. Ignore fulfilled requests.
2. Choose the highest `getPriority()` value.
3. At equal priority, choose the oldest request date.

Fulfillment policy

Use only AVAILABLE, unexpired units with exactly the same ABO and Rh type.

If the full quantity is unavailable, change nothing. No partial fulfillment is allowed.

Persistence is readable and recoverable

donors.txt

Identity, eligibility data, blood type, and optional last donation date.

blood_units.txt

Donor link, donation and expiry dates, type, and unit status.

requests.txt

Kind field restores the correct RegularRequest or EmergencyRequest subclass.

Safe loading

Missing files mean empty data; malformed rows report the exact line and stop startup.

Tests verify boundaries and state changes

Automated with plain Java assertions

The suite covers eligibility, priority, atomic matching, persistence, safe editing, dashboard counts, and headless refresh of all five Swing panels.

ALL

Controller + UI assertions

17 → 61

Age boundaries tested

0 PARTIAL

Failed matches mutate
nothing

Live demo

Review the dashboard
Process the emergency
Confirm save + restart

